IT-IM 05 Reviewer

DATA WAREHOUSING AND DATA MINING UNIT 1 – PART 2: DATA WAREHOUSE ARCHITECTURE

Data Warehouse Architecture

- **Normalized Approach**
- **Dimensional Approach**

Normalized Approach

- Based on the relational data model
- Utilizes normalized table to reduce redundancy and ensure integrity
- Ideal for:
  - When consistency and accuracy of data need to be preserved
  - Supporting complex data transformations
  - Complying on strict data quality standards

Inmon Architecture

- Developed by Bill Inmon
- Focuses on a centralized and integrated approach in data warehousing

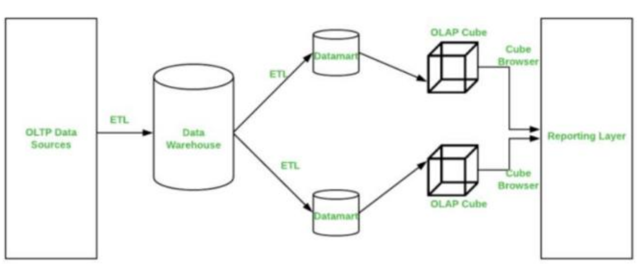
Enterprise Data Warehouse (EDW)

- a centralized repository that stores integrated and subject-oriented data

Top-down approach

- starts with EDW then into data marts

Inmon Model



Pros and Cons

+ More proficient, dependable, and scalable

- May require more complex queries, longer processing time, and higher technical skills

Reasons for Normalization

- Data Integrity
- Flexibility and Scalability
- Integration
- Reduced Data Anomalies

Dimensional Approach

Dimensional Data Warehouse

- Consists of facts tables and dimension tables
- Fact tables: store quantitative measures
- Dimension tables: store descriptive attributes
- Ideal for:
  - fast and flexible reporting
  - ad-hoc analysis
  - data visualization

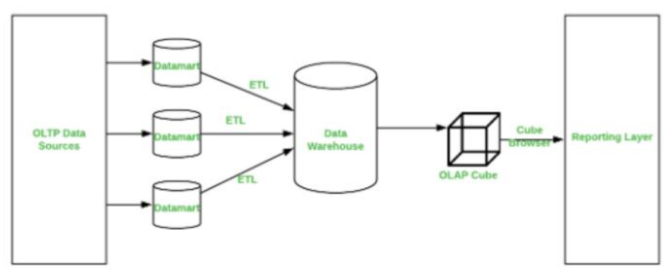
Kimball Architecture

- Developed by Ralph Kimball
- Focuses on organizing data into facts and dimensions (**dimension model**)

Bottom-up approach

- Starts with individual data marts then integrating them into an enterprise data warehouse
- Star, Snowflake, Galaxy Schema

Kimball Model



Pros and Cons

+ Easier to comprehend, utilize, and maintain

- May require more data storage and there is a possibility for data duplication and decrease data

Reasons for Denormalization

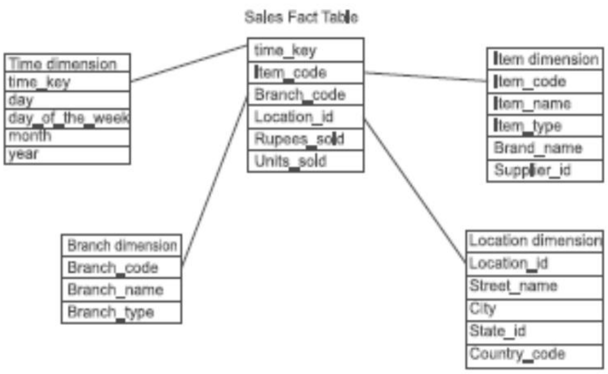
- Query Performance
- User-Friendly
- Reduced Join Complexity
- Analytical Efficiency

Schema

- Star Schema
- Snowflake Schema
- Galaxy Schema

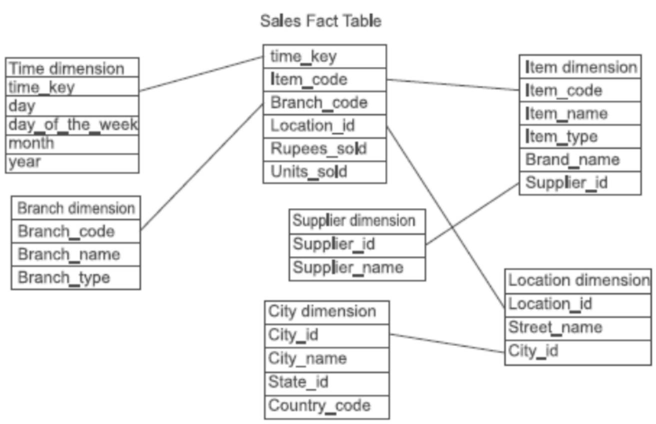
Star Schema

- Simplest schema
- Fact table is in the center
- Dimension tables are the nodes of the star
- Usually, fact tables are normalized while dimensions are denormalized



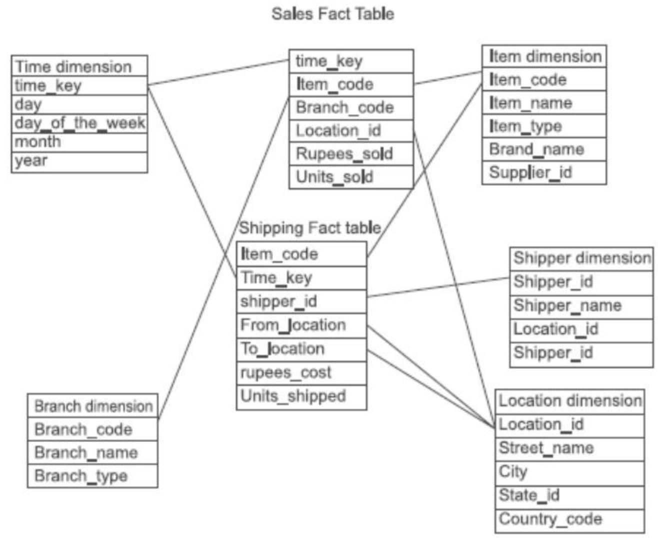
Snowflake Schema

- The main difference between the star schema and the snowflake schema is that in snowflake, there are dimension tables that are normalized.



Galaxy Schema

- The main difference between snowflake and galaxy is that galaxy consists of multiple fact tables



Comparison

Aspect	Inmon (Normalized)	Kimball (Denormalized)
Schema Design	Centralized, normalized schema	Decentralized, dimensional schema (star/snowflake)
Data Redundancy	Low, minimal redundancy	High, intentional redundancy
Data Integrity	High, due to normalization	Moderate, managed through ETL processes
Query Performance	May be slower due to complex joins	Optimized for high performance
Ease of Use	Requires more technical understanding	User-friendly, intuitive for business users
Flexibility	High, easier to adapt to new requirements	Moderate, designed for specific business processes

Source: https://www.linkedin.com/pulse/understanding-kimball-inmon-data-warehouse-omar-khaled

Scenarios

Inmon Approach	Kimball Approach
Centralized Data Warehouse: A retail company integrates all sales, customer, and inventory data into a centralized, normalized warehouse.	**Sales Data Mart:** The same retail company starts with a sales data mart using a star schema to provide quick insights into sales performance.
Data Marts: Specific data marts are created for departments like sales, marketing, and inventory, each drawing from the central warehouse.	**Customer and Inventory Data Marts:** Additional data marts are created for customer and inventory analysis, each designed for performance and ease of use.

DATA WAREHOUSING AND DATA MINING UNIT 2 – SUPPLEMENT – NORMALIZATION REVIEW

Normalization

- is the process of identifying and structuring the attributes in order to minimize or avoid data redundancies and anomalies.
- comes in series of forms:
  - First Normal Form (1NF)
  - Second Normal Form (2NF)
  - Third Normal Form (3NF)
- Main objective of normalization is to avoid anomalies
  - Insertion anomaly
  - Update anomaly
  - Deletion anomaly

First Normal Form (1NF)

- Characteristics:
  - No multi-valued attribute
  - No repeating groups
  - Primary key / key attributes are identified
  - Others:
    - Attribute domain should not change
    - Unique names for columns and attributes
    -

Example 1 (1NF)

Unnormalized

TeacherNo	TeacherName	Subject
1	A	CC101, CC100
2	B	CC100, ITNET01

1NF

TeacherNo	TeacherName	Subject
1	A	CC101
1	A	CC100
2	B	CC100
2	B	ITNET01

TeacherNo	TeacherName	Subject

Second Normal Form (2NF)

- Characteristics:
  - Must be in 1NF
  - No partial dependencies
    - Partial dependencies exist a non-key attribute is dependent only on a portion of the primary key

Example (2NF)

- Suppose we have this table in 1NF. How do we convert it into 2NF?

TeacherNo	TeacherName	TeacherAddress	Dept	Dean	SubjectCode	SubjectDesc	Units	SchedNo	Time	Day	Type
1	A	Cab City	CICT	Dr. IM	CC101	Computer Programming Fundamentals	3	1	9:00 – 11:00	Monday	Lec
1	A	Cab City	CICT	Dr. IM	CC100	Introduction to Computing	3	2	9:00 – 11:00	Tuesday	Lec
2	B	Cab City	CICT	Dr. IM	CC100	Introduction to Computing	3	3	1:00 – 3:00	Monday	Lec
2	B	Cab City	CICT	Dr. IM	ITNET01	Introduction to Networks	3	4	8:00 – 10:00	Tuesday	Lec

1NF – Teacher Table

TeacherNo	TeacherName	TeacherAddress	Dept	Dean	SubjectCode	SubjectDesc	Units	SchedNo	Time	Day	Type

TEACHER

TeacherNo	TeacherName	TeacherAddress	Dept	Dean

SUBJECT

SubjectCode	SubjectDesc	Units

TEACHING_SCHEDULE

SchedNo	Time	Day	Type

From here, we can also identify the foreign keys that are responsible for referential integrity.

In this example, Teaching_Schedule can use TeacherNo and SubjectCode to be able to identify what subject the schedule is for and who will be teaching it

TEACHER

TeacherNo	TeacherName	TeacherAddress	Dept	Dean

SUBJECT

SubjectCode	SubjectDesc	Units

TEACHING_SCHEDULE

SchedNo	Time	Day	Type	TeacherNo	SubjectCode

Third Normal Form (3NF)

- Characteristics:
  - Must be in 2NF
  - No transitive dependencies
    - Transitive dependencies exist when a non-key attribute is determining another non key attribute

Example (3NF)

TEACHER

TeacherNo	TeacherName	TeacherAddress	Dept	Dean

Dept is a non-key attribute that determines Dean which is another non-key attribute

SUBJECT

SubjectCode	SubjectDesc	Units

TEACHING_SCHEDULE

SchedNo	Time	Day	Type	TeacherNo	SubjectCode

TEACHER

TeacherNo	TeacherName	TeacherAddress	Dept

To remove the transitive dependency, we will create a new table Department.

Department

Dept	Dean

SUBJECT

SubjectCode	SubjectDesc	Units

TEACHING_SCHEDULE

SchedNo	Time	Day	Type	TeacherNo	SubjectCode

Dept as foreign key in Teacher table

Scenario: What if there's a sudden change and it has been brought up that there can be a possible anomaly. Hence, your team wants to set something more unique such as DeptNo as PK.

If you do that, you need to double check if it satisfies 1NF up until your 3NF. If no conflict, update the Teacher table too and make the foreign key as DeptNo

DEPARTMENT

<u>DeptNo</u>	Dept	Dean

TEACHER

<u>TeacherNo</u>	TeacherName	TeacherAddress	DeptNo

Final output

TEACHER

<u>TeacherNo</u>	TeacherName	TeacherAddress	DeptNo

DEPARTMENT

<u>DeptNo</u>	Dept	Dean

SUBJECT

<u>SubjectCode</u>	SubjectDesc	Units

TEACHING_SCHEDULE

<u>SchedNo</u>	Time	Day	Type	TeacherNo	SubjectCode

From this final output, we can now create the relational schema with normalized tables.

Summary

No multi valued attributes

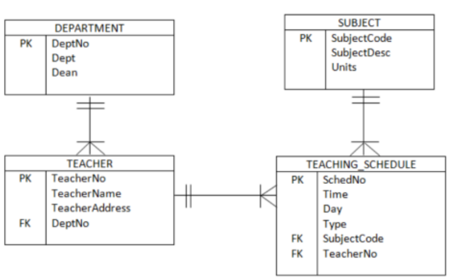
No partial dependencies

No transitive dependencies

Relational Schema

- A **schema** is a logical grouping of database objects related to one another and becomes a guide on how the database is constructed.
- It is the blueprint of a database

Schema



Per our schema:
✓ Each table has primary key which is unique across all records

✓ Related tables are connected through the foreign keys

✓ One-to-many relationships exist in tables

Schema to Table

Department

<u>DeptNo</u>	Dept	Dean

Subject

<u>SubjectCode</u>	SubjectDesc	Units

Assume that we did some coding create database and tables, from model/schema, it will become the sample table structures.

Teacher

<u>TeacherNo</u>	TeacherName	TeacherAddress	DeptNo

Teaching Schedule

<u>SchedNo</u>	Time	Day	Type	SubjectCode	TeacherNo

Department

<u>DeptNo</u>	Dept	Dean
1	A	Dr. Dean

If we are going to visualize the table with sample contents, we have to consider the columns we have defined as keys.

Subject

<u>SubjectCode</u>	SubjectDesc	Units
CC104	Information Management	3

Teacher

<u>TeacherNo</u>	TeacherName	TeacherAddress	DeptNo
123	B	Cabanatuan City	1

Teaching Schedule

<u>SchedNo</u>	Time	Day	Type	SubjectCode	TeacherNo
1	10:00 – 12:00	Monday	Lecture	CC104	123

Notice that the primary key DeptNo from Department table is a foreign key in the Teacher table.

Following the concept of foreign keys, the value of DeptNo in the Teacher table should be something that exists in the Department table.

Same goes for the TeachingSchedule and its foreign keys.

The SubjectCode and TeacherNo values should exist in their respective tables before you can input them in the TeachingSchedule table.